分治式设计驱动开发

DDD-Methodology 技术白皮书

基于 pi 与 pi-subagents 的方法论工具包

版本 v2.21（仓库 v2.21.0） · 2026 年 11 月 2 日

DDD-Methodology 项目组

摘要

DDD-Methodology（分治式设计驱动开发）是一套面向多智能体软件开发的方法论工具包，基于 pi 运行时与 pi-subagents 子代理框架实现。它针对的核心问题是：单一巨型设计文档使子代理上下文过载，实现随之偏离设计。方法论给出三条解法——文档随架构分治、上下文随职责分治、缺陷随层级路由，并以十二个固定职责子代理、九阶段固定编排、八道质量门禁与缺陷四级路由构成完整交付流水线。在工程机制层面，方法论以目录同构的分布式设计文档与签名级接口契约为冻结锚点，以契约指纹、阈值粒度、触碰即治理与声明面审计维持结构与契约的长期一致性；以声明式门禁、运行时守卫与门禁脚本构成三层机械约束，使关键规则不依赖提示词自觉。本白皮书系统阐述该方法论的问题定位、总体架构、流程与角色、门禁与路由、设计与变更治理、声明面与债务治理、机械约束、存量重建、绩效度量及工程交付形态，供工程团队评估、部署与裁剪参考。

关键词：多智能体开发；设计驱动开发；上下文分治；质量门禁；缺陷四级路由；契约冻结

目　录

摘要.....2
1　引言.....6
1.1　背景与问题.....6
1.2　解法总纲.....6
1.3　设计原则.....7
1.4　术语约定.....7
2　执行模型与总体流程.....8
2.1　执行模型.....8
2.2　九阶段流程.....8
2.3　知识分层与条件加载.....9
2.4　Workflow 加速档.....10
3　角色体系与派发纪律.....12
3.1　十二个职责子代理.....12
3.2　交接链与根会话职责.....13
3.3　派发判定.....13
3.4　派发纪律与合法拒绝.....14
4　质量门禁体系.....16
4.1　门禁总则与证据源.....16
4.2　八道门禁.....16
4.3　门禁记录与机读行.....17
5　缺陷四级路由.....19
5.1　判定表.....19
5.2　自检顺序与升级语义.....19
6　设计治理机制.....21
6.1　粒度与独立性.....21

6.2 接口先行与冻结.....22
6.3 并行实现与集成.....22
6.4 实现与测试分离.....23
6.5 测试阻塞防护.....23
7 变更管理.....25
7.1 需求变更.....25
7.2 二次开发.....25
7.3 渐进式改造.....25
8 声明面治理.....27
8.1 声明面可验证与修剪.....27
8.2 存量债务的登记与清偿.....28
8.3 结构健康度量.....29
9 机械约束与演进治理.....31
9.1 两层机械承载.....31
9.2 门禁脚本与消费契约.....31
9.3 契约指纹与变更类型.....32
9.4 触碰即治理与等价性验证.....33
9.5 项目级指令文件的维护.....33
9.6 本仓库的文档同步链.....34
10 存量重建与定向重构.....35
10.1 触发条件与前置检查.....35
10.2 价值锚点与重建流程.....35
10.3 废弃声明与等价性验证.....36
10.4 防堆叠约束与范围限定.....37
11 绩效度量.....38
11.1 度量前提.....38
11.2 度量体系.....38

11.3 消费方闭环 39
12 工程交付形态 40
12.1 仓库构成 40
12.2 安装与卸载 40
12.3 模型配比 41
12.4 运行时激活边界 41
13 适用性与边界 42
13.1 适用场景 42
13.2 方法论仓库自身的适用边界 42
13.3 已知边界与演进 42
结语 44
术语表 45
参考资料 46

1 引言

1.1 背景与问题

近年来，以大语言模型为执行主体的多智能体软件开发正在成为现实的工程形态：根会话承担调度与验收，具体的设计与实现工作交由职责单一的子代理完成。子代理的上下文窗口有限且相互隔离，设计信息如何在代理之间低损耗地传递，直接决定最终产物与设计意图的偏差程度。在这一形态下，传统上服务于人类工程师的「单一设计文档」实践遇到了新的失效模式。

这一失效模式就是单一巨型设计文档：全部设计被压缩进一份长文档，任何实现类子代理都必须整卷加载。上下文过载带来两类后果。其一，关键约束在长上下文中被稀释，代理注意力分散，细节丢失率随文档长度上升，实现随之偏离设计。其二，文档更新成本高昂，局部修订牵一发而动全身，设计文档与代码实现逐渐脱节；当偏离发生时也缺乏分级处置手段，小问题触发大回退，修复成本失控。

1.2 解法总纲

针对上述问题，方法论给出三条解法：

（1）文档随架构分治--每个有实现内容的目录持有一份 DESIGN 设计文档，设计文档树与代码目录树同构，子代理只读就近设计，不再整卷加载；

（2）上下文随职责分治--代理只加载本阶段所需知识，规范按域拆分、按需加载，知识以代理侧内联、流程内联标注与规范深读三层送达；

（3）缺陷随层级路由--测试级与实现级问题就地修复，只有设计级及以上问题才回设计层，避免小问题触发大回退。

1.3 设计原则

方法论的全部规则建立在三条设计原则之上，其唯一权威定义处在规范「问题与解法」节：

（1）判据就近权威单一——同一事实只在一处定义，其余位置只留指针，判据与执行者就近；

（2）内联即副本且副本受机械校验——内联摘要是权威的副本，副本漂移由机械校验与文档巡检拦截，冲突时以规范为准；

（3）显式优于隐含——约定、时序与边界须显式声明，未声明不得推定。

这三条原则同时施用于方法论自身：规范按域拆分并以索引承载指针，内联副本由 `ref-check` 等机械校验看护，所有判据写成 Agent 可直接执行的形态。

1.4 术语约定

本白皮书名称中的 DDD 指 Design-Driven Development（设计驱动开发）并施以分治（divide-and-conquer）策略，与软件工程中的领域驱动设计（Domain-Driven Design）无涉。全文所称「规范」指仓库 skill/ddd-methodology/references/ 下的规范文件集，为方法论唯一权威源；「根会话」指持有调度权的 pi 主会话；「子代理」指经 pi-subagents 派发的职责代理；「被开发项目」指按本方法论开发的目标应用项目。白皮书为导览性文档，若与规范文件集存在表述差异，以规范为准。

2 执行模型与总体流程

2.1 执行模型

方法论采用固定编排与只读写二分的执行模型，不采用按自主度连续分级的设计。编排由角色表与交接链固定；工具集按写入与只读二分，只读代理不持写权限，写入代理不承担审计与巡检；代理的自主范围由职责、工具集与越界升级机制三者界定，三者之外无自主空间。需要澄清的是，该执行模型与渐进式改造的 L1 至 L5 分级是两个不同的量：改造分级衡量单次改动的范围，与代理自主度无关，两者名称相近但语义不同，不得混用，也不得以改造级别推定代理自主度。

2.2 九阶段流程

开发主流程为九阶段固定编排：阶段间以门禁衔接，未过门退回上一代理；接口冻结作为独立阶段插在分解设计之后、实现之前。表 1 给出各阶段的承接代理、核心产出与门禁归属。

表 1 九阶段开发流程

序号	阶段	承接代理	核心产出	门
1	需求采集	ddd-requirements	REQUIREMENTS.md，含与用户逐条确认与新建或二次开发判定	G1
2	验收标准	ddd-acceptance	AC.md，需求转为可执行验收标准	G2
3	架构设计	ddd-architect	ARCHITECTURE.md 与项目级指令文件 AGENTS.md 初版	G3
4	分解设计	ddd-decomposer	目录同构设计文档树与签名级接口契约	G4

序号	阶段	承接代理	核心产出	门
5	接口冻结	ddd-interface-reviewer（只读）	接口评审报告，独立交叉评审	G4.5
6	实现‖测试规划	ddd-implementer 与 ddd-test-planner	按就近 DESIGN 的实现与单元测试、测试计划树	G5
7	集成收口	ddd-fixer	集成收口结论与结构治理回执	G5-INT
8	测试执行	ddd-tester	测试报告与缺陷路由单	—
9	声明面审计	ddd-auditor（只读）	审计报告与修剪清单，含恶化项	G6

过门时按模板库的门禁检查单填写并写入项目 docs/gates/ 目录，每道门一份记录文件，携带门号、过门轮次与结论的 GATE 机读行，作为绩效度量的采集来源。门禁体系的完整定义见第 4 章。

2.3 知识分层与条件加载

知识按三层送达：第一层是代理侧内联，关键规则内联在各代理提示词正文，派发即完整送达，不受上下文压缩影响；第二层是流程内联标注，根会话只在流程各步标注的时机加载对应章节；第三层是规范深读，完整规则在规范文件集中，由 skill 广告与按需读取。三层有确定的送达顺序，内联内容是规范的副本，副本与规范冲突时以规范为准，副本漂移由机械校验与文档巡检拦截。「禁止整卷加载」只约束子代理，单份模板或规范文件可按需读取。

条件加载集中于一张判据自足的对照表，命中即追加对应加载：项目为二次开发时加载「二次开发」节；本阶段产出改造影响评估时加载「渐进式改造」节；产出或修订 DESIGN 时加载「粒度与独立性」节与契约指纹条；涉及

接口冻结或指纹变化时加载「结构与契约的演进治理」节；本阶段为集成收口或审计时加载「结构与注释的机械证据」节；出现定级争议时派 ddd-arbiter 而不追加加载。未命中任何条件时不追加加载。项目类型由 ddd-requirements 判定：目录内无既有代码与配置为新建，有既有实现为二次开发；ddd-architect 另按项目是否已有自身规范文档决定是否先做现状分析，两个判据用途不同，不得互相替代。

2.4 Workflow 加速档

需求明确且要求一键跑完全链时，可经 workflow 加速档自动推进九阶段，止于声明面审计，审计报告的落盘仍由根会话执行。加速档在 C 与 D 级缺陷上自动修复重测，A 与 B 级中断返回本会话；接口评审结论非 pass 时同样中断，由根会话决定退回分解设计或修复后重跑。进度写入项目 docs/.ddd-state.json，阶段键集合为 requirements、acceptance、architect、decomposer、interface-review、implement-plan、g5-int、test-execute、audit；JSON 损坏时中断续跑并交根会话，不得当作全新启动。前置条件为加速档用到的 ddd-* 代理已安装，安装状态自检用安装脚本的 --check，模型配比状态自检用 apply-models.js status，两者职责不同，不得混用。加速档参数见表 2，其中手工派发链上的 ddd-doc-sync 与 ddd-arbiter 不在加速档派发范围内。

表 2　加速档参数

参数	默认值	下界	说明
root	无（必填）	无	项目根绝对路径
maxFixRounds	2	无	C 与 D 级缺陷自动修复轮数上限

参数	默认值	下界	说明
stallTimeoutMs	300000	30000	阶段停滞判定超时（毫秒）

3 角色体系与派发纪律

3.1 十二个职责子代理

方法论定义十二个固定职责子代理，槽位号仅作流程位置与排序标识，跨文档引用一律用代理名，代理名是唯一键。其中接口评审者、仲裁者、文档同步守护者与审计者为只读代理，不持写权限；其余为写入代理。清单见表 3。

表 3 职责子代理清单

槽位	代理	职责	产出
1	ddd-requirements	细化需求，与用户确认对齐，判定新建或二次开发	REQUIREMENTS.md
2	ddd-acceptance	需求转可执行验收标准	AC.md
3	ddd-architect	技术环境、目录规划、横切规范、基础能力清单、改造影响评估	ARCHITECTURE.md
4	ddd-decomposer	目录同构设计文档树，签名级接口契约	docs/design/**/DESIGN.md
4.5	ddd-interface-reviewer	接口契约独立交叉评审（只读）	接口评审报告
5	ddd-implementer	按就近设计实现代码与单元测试	各目录代码与测试
6	ddd-test-planner	制定单元测试计划树	docs/test/**/TEST_PLAN.md
7	ddd-fixer	集成收口与按路由单修 C 级代码或 D 级计划	修复清单与约定遵循自检
8	ddd-tester	执行测试，缺陷定级路由	测试报告与缺陷路由单
9	ddd-arbiter	定级争议独立仲裁，路由知识内嵌（只读）	仲裁裁决单
10	ddd-doc-sync	文档一致性巡检（只读）	漂移清单

槽位	代理	职责	产出
11	ddd-auditor	声明面与死代码审计，可验证性与修剪（只读）	审计报告与修剪清单

3.2 交接链与根会话职责

交接链为：需求 → 验收 → 架构 → 分解设计 → 接口冻结 → 实现‖测试规划 → 集成收口 → 测试执行 → 声明面审计。其中测试执行与实现永不同段。根会话保留调度、仲裁、用户裁决与最终验收权，不承担实现类工作：派 ddd-arbiter 处理定级争议，仅最终裁决与升级用户由根会话决定，同一缺陷超两轮强制升级用户。审计代理为只读，其报告由根会话按报告给出的建议归档名落盘至项目 docs/audits/ 目录，作为下次修剪清单回验的输入。迭代收尾时，根会话核验项目级指令文件并做适应性更新、按绩效度量节汇总本轮度量写入 CHANGELOG.md、更新债务登记簿的当前版本并录入新增债务。

3.3 派发判定

派发前先判任务性质，第一问是这个任务是否只读。目的只是获取信息、无专属产出物的探查型只读任务（探查、调研、方案比选、环境勘察、资料检索）一律派 scout，不再查表；产出接口评审报告、仲裁裁决单、巡检报告、审计报告等专属产物的职责型只读任务，按产出物查派发表派对应只读代理；非只读任务同样查表。表 4 为产出物到承接者的对照。

表 4　产出物与承接者对照

产出物	承接者
需求文档	ddd-requirements
验收标准	ddd-acceptance

产出物	承接者
架构文档	ddd-architect
分解设计与 DESIGN 修订	ddd-decomposer
接口评审报告	ddd-interface-reviewer
代码与单元测试	ddd-implementer
测试计划	ddd-test-planner
测试报告与缺陷路由单	ddd-tester
缺陷修复与集成收口	ddd-fixer
仲裁裁决单	ddd-arbiter
文档巡检报告	ddd-doc-sync
审计报告	ddd-auditor
代码评审 / 无匹配产出物	reviewer / 根会话直接完成

3.4 派发纪律与合法拒绝

每次派发须给出七项要素：目标、cwd、权限边界、相关文档路径、验收标准、期望输出格式与停止条件。实现者任务须列明其负责目录的 DESIGN 路径，禁止要求整卷加载；多模块按互斥接缝并行，一个接缝一个 writer。派发纪律有三条禁令：

（1）只读任务不得派给任何具备写权限的 ddd-* 代理，即使任务看上去很小；

（2）承接者收到不符合自身职责的任务时停止并报告「派发类型不匹配」，注明缺失的前置输入，不得改换任务性质后推进；

（3）该拒绝导致该次子任务验收判 rejected，属预期信号而非代理异常，根会话据此改派或补足前置输入，不得要求执行者将就。

scout 与 reviewer 由 pi-subagents 随带，不在十二个代理清单内，派发时使用准确名称且不加 ddd- 前缀。加速档各阶段的任务文本由固定模板给出且输入完整，不触发合法拒绝情形。

4 质量门禁体系

4.1 门禁总则与证据源

流程按门禁表的「序」列推进，未过门退回上一代理并记录于项目 docs/gates/ 目录；门号只是标识，序号才是唯一顺序依据。门禁元规则两条：默认规则是规范中可客观判定的条款均已挂门，门禁表即其归属索引；例外规则是少数指导性条款未挂门，不得作为验收依据（如「问题与解法」节的三条解法陈述、并行前提段、二次开发的现状分析判断、适用边界声明与执行模型段）。清单外若发现某条本应强制却未挂门的条款，处理方式是补挂门而非降为建议，判断标准是该条款被违反时能否客观判定。

证据源只允许三类：脚本命令，即可复现的机械证据，须写明具体命令；指名代理，即判断型证据，须写明代理名与判据；运行时守卫，即由 pi 扩展在工具调用或收尾时机械判定并阻断，须写明扩展文件与判定编号。脚本与运行时守卫属机械型，指名代理属判断型；G1、G2、G3 及 G6 的部分子项为判断型，门禁并非全门机械。

4.2 八道门禁

表 5 汇总八道门禁的位置、退出条件与证据源形态。退出条件为摘要表述，完整判据以规范「质量门禁」节门禁表为准。

表 5 八道门禁总表

门	位置	退出条件（摘要）	证据源
G1	需求→验收	需求无待确认项，已与用户对齐	判断型：ddd-requirements 自查
G2	验收→架构	每条 AC 可客观判定，AC 与需求编号双向可追溯	判断型：ddd-acceptance 自查

门	位置	退出条件（摘要）	证据源
G3	架构→分解	目录树完整，职责声明齐备，横切规范集中，基础能力消费方非空，改造影响评估与树根路径声明，项目级指令文件初版产出	判断型：ddd-architect 自查
G4	分解→接口冻结	文档树覆盖全部实现目录，模块级 AC 之和覆盖总 AC，无悬空接口引用，契约指纹已记录	机械型：trace-check.js
G4.5	接口冻结	结论行为 VERDICT: pass，必须修复项为零，契约指纹已记录；冻结后接口只能经 B 级变更修改	判断型：ddd-interface-reviewer
G5	实现→测试执行	各接缝自测通过且失败项逐项拆分归属，TEST_PLAN 覆盖全部模块级 AC，高风险用例已桩化，指纹变化时用例未逆向缩减	机械型：trace-check.js
G5-INT	集成收口	构建通过，自测与治理回执入完成报告，实现粒度与设计预判一致，超限文件已治理，本阶段未执行全量测试	机械型：code-health.js、governance-check.js；判断型：ddd-fixer
G6	测试→交付	测试全绿或未过项已定级且未超轮，审计无待修剪项且报告已归档，指纹与变更类型一致，恶化项有接受理由，指令文件核验，债务校验通过	判断型：ddd-auditor；机械型：debt-check.js

4.3 门禁记录与机读行

门禁记录由根会话写入项目 docs/gates/ 目录，文件名为 G{门号}-记录.md，须含机读行 GATE: {"id":"G{门号}","round":<过门轮次>,"verdict":"pass|fail"}。round 为过门轮次，首次过门为一，因退回而重过递增；verdict 取 pass（通过）或 fail（退回）。该机读行是八道门一

次通过率与过门轮次两项度量的唯一来源，由度量聚合脚本读取采集。过门时按模板库的门禁检查单填写，检查项与证据源列同源，不得各写一遍。

需要说明的是，结构健康度量首版仅报告不作阻断：G5-INT 的证据源中 code-health.js 恒退出零，真正阻断者为 governance-check 的 verdict 与集成收口的治理条件；G6 要求的是恶化项逐项给出接受理由或已定级改造，而非恶化项为零。脚本类证据的规范调用形式为 node ~/.pi/agent/skills/ddd-methodology/scripts/<脚本名>.js <项目根>，脚本随 skill 安装在固定位置，不由目标项目提供。

5 缺陷四级路由

5.1 判定表

缺陷按四列判定表定级，命中即停；未命中任一级特征时默认取最低适用级别。该路由把「小问题就地修、设计级以上才回设计层」落成可执行判据，见表 6。

表 6 缺陷四级判定表

级别	定义	自检问题	处理
D 测试级	计划漏场景、断言错或覆盖缺口	计划是否遗漏或断言错误	ddd-fixer 修计划
C 实现级	实现偏离设计	代码与就近 DESIGN 何处不一致	ddd-fixer 修码，不改设计
B 设计级	设计矛盾、边界不清、接口不可实现	实现者是做不出来而非没做对	重新走分解设计并同步 AC
A 需求级	设计正确执行仍不满足总 AC	是否需求本身矛盾或缺口	回需求采集，必须经用户确认

5.2 自检顺序与升级语义

自检按固定顺序五步执行：先自问该缺陷是否属 D 级；D 级不成立时自问 C 级；C 级不成立时自问 B 级；B 级不成立时自问 A 级；命中即停，取该级处理动作。只有 B 级及以上回设计层。

两轮升级是贯穿方法论的一致语义：同一缺陷轮回修复超过两轮仍未解决，强制升级用户，仲裁计入轮回；存量债务连续两轮超期、同一根因分类连续两版登记为未清，同样触发升级。测试报告与缺陷路由单由 ddd-tester 归档至项目 docs/reports/ 目录，测试报告命名 test-<版本>-r<轮次>.md，缺

陷路由单命名 defect-<版本>-r<轮次>-<缺陷ID>.md，同一轮唯一定位、多轮不覆盖；归档件保留报告末行 DEFECTS 机读行与缺陷路由单的轮回计数字段，作为返工轮次分布与缺陷四级分布的采集来源。

6 设计治理机制

6.1 粒度与独立性

粒度阈值是结构与文档治理的基础判据，唯一权威定义处在规范「粒度与独立性」节，其余文档引用阈值时须与该锚点一致，由 ref-check 校验。接近上限指规模超过上限的零点八倍，口径与 code-health.js 的接近上限判定一致。阈值按档位分配承载，硬拦靠运行时守卫，度量靠脚本，拆解判定靠分解设计者，抽检靠审计者，见表 7。

表 7 粒度阈值与承载

阈值	数值	承载体
代码文件	500 行	运行时守卫硬拦（H-1），显式声明运行时例外者放行
基础或核心模块	300 行	code-health.js 脚本度量
文档	200 行	脚本度量，配 ddd-decomposer 拆解判定与 ddd-auditor 抽检
设计文档契约数	10 项	脚本度量，配 ddd-decomposer 拆解判定与 ddd-auditor 抽检

文档粒度：同构场景达到三个，或单一 DESIGN 文档超过二百行或十项契约，必须拆解为分类文件或子场景子目录，父级只留共性契约与子场景索引，不重复子场景细节。代码粒度：单文件超过五百行、基础或核心模块超过三百行，或承载两个及以上独立可测能力，必须拆解；被多模块依赖的基础能力独立成文件并配套独立单元测试；DESIGN 须预判实现文件粒度与职责边界，实现不得合并已拆解文件。运行时例外允许受平台约束无法拆解的单文件视为合规，须同时满足四个判定项：平台受限、文件头显式声明、单一能力域、未声明不适用。

独立性要求模块、类、函数、变量保持相对独立，避免调用顺序影响结果：函数优先无副作用；模块间只经冻结接口通信；类构造完成即可用；变量缩小作用域并优先不可变。契约未标注时序依赖即视为顺序无关，实现不得引入隐式顺序，独立单元测试以乱序调用验证顺序无关性。

6.2 接口先行与冻结

接口在分解设计期写成签名级契约，七要素齐备：签名、参数、返回值、错误语义、时序依赖与副作用、消费方模块、调用示例。契约表置于文档前部，调用方只读契约即可集成。无消费方的接口不得写入契约，也不得引入将来可能用到的预留声明。

冻结的产物是契约指纹快照，不是一个状态词：指纹未变即结构可自由改动，无需审批，但仍须遵守粒度预判与触碰即治理义务，无需审批不等于无需设计预判；指纹变化即行为变更，必须递增版本并走 B 级路由同步 AC，不得静默修改。接口冻结由 ddd-interface-reviewer 独立评审，检查项与判据以其九项检查表为唯一准据，任一 fail 项即判 fail，无 fail 而有 conditional 项时判 conditional；结论以 VERDICT: pass|conditional|fail 落行，conditional 属非 pass，一律阻断冻结。指纹的构成与算法见 9.3 节。

6.3 并行实现与集成

实现阶段按依赖约束图划分互斥接缝，一个接缝一个 writer，同一文件任一时刻只有一个写入者。并行前提三条须逐条成立：接缝无循环依赖；设计与测试计划齐备；横切规范已集中定义。接缝间只通过 DESIGN 声明的接口耦合，需要改他人接口时停止并报 B 级。

分工规则与交付责任成对适用，只给前者会得到各接缝都交付了但无人对结果负责：他人接缝与中间态不修，跨接缝问题只经 DESIGN 声明的接口沟通；本域停手交付时必须全绿，发现失败项先判定归属，属本域者修完方可交付，属他域者须显式登记归属并报根会话，不得仅以非本域为由交付。失败项不得整体归因于他人，混合的红必须逐项拆开并标注归属，否则视为交付不合格。全部 writer 完成后由根会话收口，再触发集成检查。

6.4 实现与测试分离

测试规划可与实现并行，测试执行必须在实现收口过 G5-INT 后单独串行启动，与实现永不同段、不共用进程。G5-INT 与 C 级修复禁止执行全量测试套件，全量测试的唯一主体是 ddd-tester。该分离防止实现代理借「跑测试」之名干预全局状态，也保证测试结论来自独立主体，使缺陷路由单的定级可信。

6.5 测试阻塞防护

测试计划为每条用例标注预估时长与风险级别，并给出含超时的执行命令，五条防护规则如下：

（1）涉及网络、长循环、大数据量、外部服务或真实等待的用例必须桩化，或移入手动测试清单；

（2）执行时只照抄计划中的命令，禁止裸跑；

（3）连续两次超时强制转手动清单；

（4）手动清单由用户执行反馈，待反馈项不阻塞其余缺陷路由；

（5）报告区分自动通过、自动失败、手动待反馈、手动失败四态。

7 变更管理

7.1 需求变更

需求变更走版本化追加与按影响面回归，收口由声明面审计确认。用户变更需求后按五步执行：

（1）登记变更单；

（2）需求与 AC 版本化追加而非覆盖；

（3）依据 AC 双向追溯圈定受影响的 DESIGN 集合；

（4）只对受影响接缝重设计、重实现与回归；

（5）变更完成后派 ddd-auditor 做声明面审计。

7.2 二次开发

二次开发项目仅当缺少自身规范文档时才做现状分析，输出结构推断、原有准则提炼、同构约束清单与风险坑位；已有规范文档时直接对齐，新模块必须与原结构同构。是否做现状分析取决于项目是否已有自身规范文档，属设计期判断，与 ddd-requirements 的新建或二次开发类型判定是两个不同判据，用途不同，不得互相替代。

7.3 渐进式改造

追加新功能时若现有设计不合理，或结构健康度量相对上次恶化，进入改造分级。一次迭代只做一个级别，禁止跨级合并改造，见表 8。

表 8 改造分级（L1–L5）

级别	范围	处理
L1 文件级	单文件内	触碰该文件的写入者自行完成并记变更记录，无需根会话派发
L2 目录级	单目录多文件	分解设计修订 DESIGN

级别	范围	处理
L3 模块级	跨目录或契约变更	B 级设计变更走正式路由
L4 架构级	目录树或依赖方向	根会话确认后修订 ARCHITECTURE
L5 项目级	技术环境或全局规范	必须升级用户确认

定级按三步执行：从 L1 起评，范围超出该级时上移一级，命中即停，不得跳级命名。一次迭代只做一个级别的约束作用于 L2 及以上；触碰即治理义务属 L1，不受该约束。改造影响评估在架构设计期完成，结果写入 ARCHITECTURE 的改造影响评估节，是 G3 的退出条件，其中的存量债务承接列是分解设计者编制粒度预判的输入；恶化项的定级由根会话依据审计报告在 G6 后派发。存量重建不属本分级，见第 10 章。

8 声明面治理

8.1 声明面可验证与修剪

声明面治理的铁律是：不被消费的声明是危险的，因为无人验证其正确性。每条声明须同时具备消费方与验证手段，缺一不得引入。声明面包括接口契约、公开导出、能力清单、规则条款、配置项、模板字段与依赖项，执行入口为 G6，即审计报告未过则不得交付。pi 运行时契约类声明（如 worktree、control 字段与 frontmatter 字段）以冒烟检查与运行时观察为验证手段，不因无法离线验证而判为纸面消费方。

审计要求四条：其一，审计类代理全量核对，不得因看似无用、低风险或成本高而跳过，对每项回答是否被消费与消费是否正确，发现无消费方声明即报缺陷，不得静默留存；其二，反向抽查不少于三项并实测消费路径真的通，专项审计可自行提高下限；其三，除清单核对外显式记录探索性审计的检索面与结论，使无发现成为可复核的结论而非缺省，不得以「清单全通过」代替本项记录；其四，同时核验变更记录当轮声称与实际产物是否相符，声称与实际不符即为缺陷，与代码缺陷同级处理。

无用代码与冗余声明必须删除，不得注释保留，不得以将来可能用到为由滞留。写入者的清理义务限于所触及文件内的连续注释代码与兼容层标记，属文件级属性；无引用文件属仓库级属性，由审计代理处理，不入写入义务。清理义务的触发点为：结构健康度量显示恶化时、G6 交付前、需求变更后、能力新增后、版本收尾、重建完成后。修剪清单在下次审计时必须逐项回验是否真的删除。审计报告由审计代理给出建议归档名（格式 G6-序号-版本，序号取 docs/audits/ 现存最大序号加一），落盘由根会话执行至项目 docs/audits/ 目录。

8.2 存量债务的登记与清偿

存量债务指承接方判定为范围外、或须先补设计预判因而未在本轮清偿的项。登记不是免责而是清偿的入口：只写出口即登记而不写入口即清偿，解耦即退化为无出口的累积。最小闭环由五件机制构成--统一登记簿、具名承接、清偿时限、超期校验与总量上限、债务历史快照与根因复发检测，另有教训回写作为清偿出口。登记簿为项目 docs/debts.json，是存量债务的唯一登记处，条目字段见表 9。

表 9 债务登记簿（docs/debts.json）条目字段

字段	含义
id	编号，形如 D-001，仓库内唯一
desc	一句话描述
registeredAt	登记时的版本
owner	承接方，须具名，即具体代理名或根会话
dueBy	应清版本，可省；省略时按登记版本加 defaultDueSpan 推导
status	open 或 closed
reason	登记理由，例如范围外、需先补设计预判
rootCause	根因分类，取五类之一
recurrence	复发记录，含 count 与 lastVersion

登记簿顶层三字段固定：currentVersion 记当前版本，maxOpen 记未清上限（固定默认十项，「约三倍」仅说明默认值由来不构成计算规则），defaultDueSpan 记默认时限跨度（默认为二，即登记版本加二、两版本内清偿）。owner 必填且具名，禁止「留待下一轮」一类无对象表述，因其无承接方等于无人负责。超期校验由 debt-check.js 机械承担，读登记簿与当前版本

逐项判定，超期即阻断交付门，连续两轮超期强制升级用户；未清超上限触发专项清偿轮，该轮不得新增功能。债务的版本跨度与缺陷的修复轮次是两个不同的量，前者按登记版本与应清版本计，后者按同一缺陷的轮回计，不得混读。

debt-check.js 每次运行维护 docs/debt-history.json 快照，history 数组按版本新增或就地更新，历史上限五十条超出截断；复发判定为机械口径——某根因分类在上一版本快照已有未清项，且当前版本新增登记含同根因条目，即判该根因复发。根因分类取五类：声明与事实脱节、单一权威被违背、机械校验有盲区、规则演进后消费方未同步、反复修补同一处。教训回写是清偿出口：清偿根因为「机械校验有盲区」的债务时，该根因须转化为新的机械校验项并在 docs/check-coverage.json 登记根因分类到校验项的映射（条目含来源债务编号），不得只改文档；映射的存在性与一致性由机械校验核对，反向完整性由审计代理核验。

8.3 结构健康度量

code-health.js 输出五类结构事实，写入项目 docs/health.json（入版本管理作为趋势证据，历史上限五十条），作为 G5-INT 的证据源与 G6 审计的输入。度量仅报告不阻断，其阻断地位由 G5-INT 的治理条件承接，度量恶化的具体项写入审计报告的必填节。五类事实见表 10，表中未列出的结构事实不由脚本输出，须由审计代理语义判定。

表 10　结构健康度量五类事实

类别	覆盖的事实	机械程度
文件规模	超阈值与接近上限的代码文件、模块、文档，契约数超限与接近上限	完全

类别	覆盖的事实	机械程度
死代码	连续注释代码块、兼容层标记、无引用文件	大致，不可达分支需 AST 故未覆盖
重复嫌疑	同名声明跳文件出现	弱信号，语义级重复需审计代理判定
注释	违禁模式与提及已不存在标识符的过时引用	部分，是否解释为什么由审计代理抽检
结构趋势	被触碰文件规模变化，超阈值文件数相对上轮增减	完全

9 机械约束与演进治理

9.1 两层机械承载

可机械判定者分两层承载：声明式门禁为主，运行时守卫为辅。声明式门禁为代理 frontmatter 的 acceptanceRole、acceptance 与派发 options 的 gate，对所有子代理生效；写入类代理显式关闭 acceptanceRole 带来的自动评审推断，独立评审由 G4.5 与 G6 承担，避免每次实现重复评审。运行时守卫由 extensions/ddd-governance.ts 在工具调用处拦截，覆盖根会话与后台子代理（前台子代理不加载 ambient 扩展），两条判定规则见表 11；未命中守卫的工具调用一律放行。

表 11　运行时守卫判定规则

编号	触发点	判定条件	动作	当前覆盖
H-1	write/edit 工具调用	估算写入后行数超 500 行	拦截，理由含文件路径、就近 DESIGN 路径与拆解要求	代码文件 500 行阈值；文件内显式声明运行时例外者放行
H-2	bash 工具调用	命令属窄口径的全量测试调用形态，且状态文件 lastStage 为 g5-int	拦截，指明全量测试的唯一主体是 ddd-tester	整仓测试命令的窄口径写法

9.2 门禁脚本与消费契约

六个门禁与度量脚本构成机械证据面，各脚本的输出形态、机读结论与退出码由消费契约表固定：输出上限一万二千字节（UTF-8 计量），超限截断明细，但末行机读结论必须保留且完整；未列入契约表的脚本不构成任何门的证据，见表 12。

表 12　脚本消费契约

脚本	输出形态	机读结论	退出码	门禁地位
trace-check.js	文本正文	末行 RESULT: pass 或 RESULT: gap	0 通过，1 存在缺口	G4 与 G5 门禁
code-health.js	文本正文	末行 RESULT: report	恒 0	仅报告，不作门禁
governance-check.js	单 JSON	verdict 字段（ok 或 blocked）	0 通过，1 阻断	G5-INT 门槛
ref-check.js	文本正文	末行 RESULT: pass 或 RESULT: drift	0 通过，1 存在漂移	本仓库文档同步链
debt-check.js	文本正文并写债务历史快照	末行 RESULT: pass 或 RESULT: overdue	0 通过，1 缺口	交付门前置校验
metrics-report.js	文本正文	末行 RESULT: report	恒 0	仅报告，不作门禁

governance-check.js 输出单项 JSON verdict，校验三项判据：每份 DESIGN.md 的契约指纹已记录且与契约段实算值一致；docs/health.json 最新一条相对上一条问题类计数未上升（问题类指超限、死代码与注释质量类，接近上限属预警、弱信号重复需语义判定，二者不参与阻断）；存在设计文档树时其根路径已在 ARCHITECTURE 声明。三者同时成立 verdict 为 ok，否则 blocked；无设计文档树时零考察，verdict 为 ok。命令路径可确定时可经 typed gate 挂载，不可用 permission 实现，因其对 bash 恒放行。

9.3 契约指纹与变更类型

契约指纹是接口冻结的机械锚点，回答「冻结的是哪一版」，算法唯一权威在规范「契约指纹」条：取 DESIGN 中标题为「对外接口与约定」的一节，自该标题起至下一个二级标题前的内容；剔除其中的契约指纹行本身；逐行去

首尾空白后以换行连接，去尾部空行；取 SHA-256 十六进制前八位。版本号由指纹前八位派生。指纹的机械校验由 trace-check 承担，门槛校验由 governance-check 承担。

每次改动须声明类型：重构为契约指纹不变、行为用例全绿、结构度量改善三者同时成立；行为变更为契约指纹变化；重建为整体替换既有实现与文档，属行为变更的特例，须附废弃清单与回溯点，等价性以 AC 全量回归验证。声明类型与实际证据不符即为缺陷。

9.4 触碰即治理与等价性验证

写入者对所触碰文件的存量结构负治理义务，不限于清理连续注释代码与兼容层标记，代码与文档同等适用，判据为粒度阈值：文件已达或接近阈值时先按就近 DESIGN 的粒度预判拆解，无法拆解方上报；拆解不得超出 DESIGN 预判，超出属设计偏离走 B 级。全部写入代理的完成报告均须附结构治理回执，记录被触碰文件的改动前后规模、是否拆解与依据的粒度预判，空节不得声明完成。本义务属 L1，不受一轮一级约束。

等价性验证由测试执行者先判定变更性质：指纹不变时验证用例集合未减少且全绿；指纹变化时核对版本已递增与 AC 已同步，未同步报 B 级。测试用例集合逆向缩减是堆叠信号。

9.5 项目级指令文件的维护

项目级指令文件为置于项目根的 AGENTS.md，按模板五节骨架产出——项目定位与技术栈、目录结构与依赖方向、本项目特有约定与禁忌（逐条带判据）、实际使用的方法论要点（含门禁与产物路径）、已知债务与遗留（含接受理由或承接去向），由运行时逐字注入各子代理的常驻上下文。初版由 ddd-

architect 在架构设计期产出，迭代收尾由根会话核验并做适应性更新；不由各写入者各自追加，避免其退化为追加式堆积。适应性重构的判据：局部过时做适应性修订；超过文档粒度阈值（二百行）、出现描述与实际不符的条目、出现无消费方的条目三条满足任一即整篇重写。文件不写方法论通用条款——复制方法论全文会形成副本，与「单一权威与就近送达」原则冲突。项目的关键规则若子代理必须看到，写入项目级指令文件或派发任务文本，不得只留在方法论本体。规则生效分四层承载，见表 13。

表 13　规则的四层承载

机制	层级	作用
全局规则	用户级	跨项目强制规则
方法论规范	方法论级	跨项目通用条款
项目级指令文件	项目级	本项目特有约定与方法论要点
派发任务文本	任务级	单次派发的判据内联

9.6　本仓库的文档同步链

方法论仓库自身以 ref-check 维护跨文档一致性，共十八项校验（C1 至 C18），覆盖语义化节引用、裸章节号、阈值常量、阶段键、代理与 skill 清单、门号、脚本名、冻结短语、守卫编号、规范索引、安装清单、登记簿字段、校验映射、行尾一致、结构树一致与安装参数一致等域；规范修订后运行 node scripts/ref-check.js . 作为提交前检查，默认只读不写盘。加 --fix 时由规范「状态文件」节的键清单生成 workflow 脚本的 stageRegistry 并校验守卫的阶段键单键归属，写盘范围限于注册表数组块，改完须复跑默认模式确认无漂移。共享校验模块位于 scripts/lib/，供各门禁、校验与度量脚本复用。

10 存量重建与定向重构

10.1 触发条件与前置检查

存量重建由用户显式触发，模型不得自动发起，须在会话中直接调用 ddd-rebuild 或 ddd-refactor skill。进入重建须至少命中一条判定信号：修补成本高于重建成本；结构健康度量持续恶化且恶化项的接受理由已穷尽；同构场景碎片累积到拆解不再能改善结构。触发时机由人工判断，落在入口，重建不由模型在常规迭代中提议并执行。

前置检查为硬约束，五步依次执行，任一步骤不通过即停止不得动工：确认版本控制系统在场且可用；确认工作区无未提交改动或已有明确的处置记录；记录回溯点标识（提交哈希、分支名与记录时间，可按需附加标记）；把回溯点标识写入重建产物并持久化。丢弃是相对旧版本的丢弃而非绝对丢失：旧版本始终可从回溯点读取，需要参照旧实现时从回溯点读取，不在案保留旧文件。

10.2 价值锚点与重建流程

价值锚点限于三类，固定表述：对外契约；经审计确认仍然成立的设计决策；仍在被消费的能力。锚点清单由根会话产出，须经用户确认后冻结，冻结后不得扩范围，锚点之外一律可弃；未确认不得进入重建执行。重建按十步固定顺序执行：前置检查与回溯点记录；现状全量盘点与锚点候选提取；用户确认并冻结锚点与范围；按冻结锚点生成新架构与新设计，整篇重写而非修订旧文档；按新设计实施，实现与测试同等重建；集成收口，核对接缝并修集成问题；执行 AC 全量回归验证；派审计代理做声明面审计；通过后执行旧产物废弃清除；重写架构基线。自载门名共五道，不入全局门禁表，见表 14。

表 14　重建五门

门	对应步骤
前置检查门	第 1 步：前置检查与回溯点记录
锚点冻结门	第 3 步：用户确认并冻结锚点与范围
接口评审门	第 4 步：按冻结锚点生成新架构与新设计
重建验收门	第 7 步：AC 全量回归验证
审计门	第 8 步：声明面审计

十步流程的发散与收敛分段明确：第 2 步现状盘点属发散段，产物为草稿不入基线，冻结后作废；第 3 步为冻结门；第 4 步起属收敛段，不得夹带范围外变更。开放与约束分界：架构方案、目录划分、技术选型、实现路径、拆分粒度与内部命名属开放区，重构者可自由选择最优形态，不受旧实现结构牵制；产物形态与字段、流程顺序与门禁、锚点继承、废弃声明、等价性验证、回溯点记录与变更记录属绝对约束区，无论方案如何选都必须按契约产出，不得自创格式或跳过门禁。

10.3　废弃声明与等价性验证

被替换的架构、代码、测试、文档与配置须逐项列入废弃清单，含原路径与替代物或替代位置；旧产物不在案保留，需要参照时从回溯点读取；清除在验证与审计通过后执行；未列入废弃清单而实际不再使用的产物，视为静默保留，属缺陷。重建属行为变更特例：契约变更逐项列入变更清单，行为等价性以 AC 全量回归验证，不得以子集用例证明等价，用例集合不得逆向缩减，缩减即堆叠信号。

10.4 防堆叠约束与范围限定

防堆叠三条是定向重构与普通增量改造的关键区别，同等适用：范围外文件不得改动，发现问题即登记为存量债务，不在本轮顺带处理；被重构范围内的产物整篇重写，不得在旧文本上追加；完成后须给出本次范围内的废弃项清单，即使为空也须显式声明为空。

重建是一次性整体操作，不受渐进式改造「一次迭代只做一个级别」的约束；定向重构是重建的局部特例，范围限定于用户指定的模块或问题，除范围限定外重建条款同等适用。重建完成后，「触碰即治理」义务照常适用于后续迭代。

11　绩效度量

11.1　度量前提

度量体系建立在三项前提上：全部取自既有产物，不引入在线遥测，不新增需要额外采集的运行期数据源；每项度量只定义计算方式与来源产物路径，源产物未产出时该项记空，不得以估算或推测值填入；每项度量须有消费方，无消费方的度量不得引入，已无消费方的度量须删除而非留存。度量本身同样受声明面铁律约束。

11.2　度量体系

度量分交付质量、结构与债务、声明面三类共九项，全部由 metrics-report.js 按规范口径聚合为文本报告与 docs/metrics.json，见表 15。缺陷密度的分母取项目源码千行数而非本轮触碰行数：触碰行数无机械来源，改用源码总行数后分母由 docs/health.json 提供，且使密度跨轮可比；机械证据覆盖率计门不计项，含任一机械型证据源的门即计入分子。

表 15　绩效度量体系

类别	度量	计算方式	来源产物
交付质量类	八道门一次通过率	首轮即通过的门数 ÷ 门总数	docs/gates/ 的 GATE 机读行
交付质量类	返工轮次分布	缺陷轮回计数分档计数	docs/reports/ 缺陷路由单
交付质量类	缺陷四级分布	按判定级别 A、B、C、D 分组计数	DEFECTS 机读行与缺陷路由单
交付质量类	缺陷密度	四级分布之和 ÷ 源码千行数	缺陷路由单与 docs/health.json
结构与债	超阈值文件数	相邻记录超阈值文件数之差	docs/health.json

类别	度量	计算方式	来源产物
务类	趋势		
结构与债务类	债务未清数与超期率	open 条数；超期未清数 ÷ 未清数	docs/debts.json 与 debt-check.js
结构与债务类	同一根因复发次数	按根因分类汇总复发计数之和与最大值	docs/debts.json
声明面类	无消费方声明占比	无消费方、纸面消费方与消费不正确三类之和 ÷ 声明总数	最新 G6 审计报告逐项表
声明面类	机械证据覆盖率	含机械型证据源的门数 ÷ 门总数	门禁表证据源列

11.3 消费方闭环

度量全部有消费方，不存在仅登记的度量：迭代收尾由根会话经 metrics-report.js 聚合各来源产物后写入 CHANGELOG.md 作为版本趋势证据；G6 审计核验结构、债务与声明面三类度量的恶化项，逐项给出接受理由或定级改造。新增度量须同时给出计算方式、来源产物路径与消费方之一，三者齐备方可引入；缺消费方的度量即使计算简单也不得引入。

12 工程交付形态

12.1 仓库构成

方法论以单一仓库交付，顶层由七部分构成：skill/ 下的三个 pi skill——ddd-methodology 主 skill（规范权威源为其 references/ 规范文件集）、ddd-rebuild 存量重建与 ddd-refactor 定向重构（后两者仅由用户显式调用）；agents/ 下的十二个代理定义（pi-subagents 格式）；extensions/ddd-governance.ts 运行时守卫；scripts/ 下的安装卸载脚本与门禁校验脚本族（含 scripts/lib/ 共享层）；profiles/models.json 按代理模型配比；global/APPEND_SYSTEM.md 全局规则区块；workflows/ddd-workflow.js 加速档脚本；docs/ 项目侧证据与归档（结构健康度量、债务登记簿、度量聚合、校验映射与改造设计冻结契约）。仓库结构明细由 STRUCTURE.md 承载并经 ref-check 的结构树校验（C17）看护。

12.2 安装与卸载

安装卸载均为全局操作，Windows 与 Linux、macOS、Git Bash 各有一组脚本，均幂等：安装将三个 skill 与随装脚本、运行时守卫扩展、模型配比记录文件（含写入前原值供精确回退）与全局规则区块写入用户 pi 目录；脚本与代理按固定文件名清单复制与移除，不影响用户自有代理与 skill。全局规则区块按标记合入用户全局文件，区块外用户自定义内容原样保留，首次安装前自动备份，卸载时若仅剩标记区块则优先从备份恢复。卸载会移除本方法论写入的模型配置。配比应用与卸载依赖仓库在场：profiles/ 与配比应用脚本不随安装复制，仓库外调用可用 --repo 指向仓库根。安装后使用方式为在任意 pi 会话中以 /skill:ddd-methodology 或「按 ddd 方法论开发」触发。

12.3 模型配比

模型名属部署政策，不写入代理定义，只由 profiles/models.json 经 apply-models.js 应用到用户设置，因此可随时更换配比而不改动代理。提供 balanced 与 quality 两档：balanced 为推荐档，契约与编码角色用契约忠实度满分的一档，其余用同族更大输出上限的一档；quality 为质量优先档，少量高风险角色换异构模型以降低同族共同盲区。应用流程默认两道校验——注册表校验（模型须已在 pi 注册）与可用性冒烟（以最小请求实测首字延迟与推理开销，空内容即视为不可用），任一不通过的模型被跳过并告警，不写入无法验证的声明；第二道可用 --skip-verify 跳过，仅在已确认模型可用时使用。配比文件内记录实测证据与理由，样本量有限，使用中出现反例应更新该表。

12.4 运行时激活边界

运行时守卫仅在含 docs/design 或 docs/gates 的项目内激活，对其他项目静默，任何异常一律放行并记警告——守卫自身的故障不得阻断正常开发。加速档状态文件 docs/.ddd-state.json 的阶段键集合与 workflow 阶段流一一对应，每阶段完成后由该阶段代理或其后继写入者合并更新，并行段仅实现者写入；JSON 损坏时中断续跑并交根会话，不得当作全新启动。

13 适用性与边界

13.1 适用场景

方法论面向以 pi 为运行时、以子代理为主要执行体的软件开发场景：新建项目走全流程九阶段；二次开发项目叠加现状分析与改造影响评估；结构性恶化的存量项目另有重建与定向重构两条显式入口。其价值主张随项目规模与代理协作深度放大——设计文档越大、代理上下文越紧张，分治与就近设计的收益越显著；反之，单人单代理的小型脚本任务引入全套门禁的收益有限，可只取其设计原则与粒度阈值作轻量参照。

13.2 方法论仓库自身的适用边界

项目门禁与交付条款面向被开发的应用项目；方法论仓库自身是规范与工具，不含应用代码，其维护只走四条链：文档一致性走 ddd-doc-sync 巡检；声明面与结构健康走 ddd-auditor 审计（证据源为 code-health）；结构治理走触碰即治理与存量的粒度、清理、重复治理；改造设计文档位于 docs/refactor/，是仓库自我迭代的冻结契约来源。G1、G2、G3、G4、G4.5 与 G5 的证据源在本仓库报不适用而非缺口——仓库无 REQUIREMENTS.md、AC.md、ARCHITECTURE.md 与设计树，这些门无对象可判；契约指纹以 DESIGN 为对象，本仓库同样报不适用。两侧的审计归档边界不得混用：仓库自身的审计记录由 CHANGELOG.md 承载，被开发项目的审计归档仍在 docs/audits/。

13.3 已知边界与演进

方法论明示若干边界：粒度阈值（代码文件五百行、基础或核心模块三百行、文档二百行、契约十项）为经验值，接近上限取零点八倍口径；模型配比

证据为单轮对照、样本量有限；结构健康度量的死代码类覆盖为大致程度，不可达分支需 AST 未覆盖，重复嫌疑为弱信号。方法论对自身施用同一套声明面原则：每项度量与校验须有消费方，无消费方者删除而非留存；内联副本漂移由机械校验拦截。这些边界随使用反馈演进，版本以 `git tag` 管理并见 CHANGELOG.md。

结语

DDD-Methodology 把「防止实现偏离设计」这一目标拆解为可执行、可校验、可度量的工程体系：以目录同构的分布式设计文档与签名级契约锚定设计意图，以固定编排的十二个子代理与八道门约束交付过程，以缺陷四级路由与两轮升级控制回退成本，以契约指纹、触碰即治理、声明面审计与债务登记簿维持长期一致性，以机械校验与运行时守卫替代对提示词自觉的依赖。方法论自身受同一套原则治理——判据就近权威单一、内联即副本且受机械校验、显式优于隐含——使其成为一套能够自我修正、持续演进的工程实践。

术语表

术语	释义
DESIGN	目录同构的分布式设计文档，承载就近实现的设计与签名级契约
契约指纹	DESIGN 契约节内容的 SHA-256 前八位哈希，接口冻结的锚点
接缝	依赖约束图上划分的互斥实现边界，一个接缝一个写入者
门禁	阶段推进的退出条件与证据源，未过门退回上一代理
缺陷四级路由	D、C、B、A 四级判定表，小问题就地修、设计级以上回设计层
声明面	接口契约、导出、能力清单、规则条款、配置、模板字段与依赖项的总称
存量债务	范围外或待补预判而未在本轮清偿的项，登记于 docs/debts.json
触碰即治理	写入者对所触碰文件的存量结构负治理义务
回溯点	重建前记录的版本控制标识，含提交哈希、分支名与时间
两轮升级	同一缺陷或债务连续两轮未决即强制升级用户的一致语义
加速档	固定阶段自动编排的 workflow 脚本，覆盖九阶段止于审计
scout 与 reviewer	pi-subagents 随带的只读代理，不属十二代理清单

参考资料

[1] DDD-Methodology 仓库 README.md：仓库定位、安装卸载与使用入口。

[2] STRUCTURE.md：仓库结构与安装内容明细。

[3] skill/ddd-methodology/references/ 规范文件集（spec.md 索引、spec-role、spec-gate、spec-design、spec-change、spec-declare、spec-govern、spec-rebuild、spec-metrics 与模板库）：方法论唯一权威源。

[4] docs/refactor/DESIGN.md：本仓库自我迭代的改造设计冻结契约。

[5] CHANGELOG.md：自 v1.0 起的变更记录与版本度量。

本白皮书对应仓库版本 v2.21.0（2026 年 11 月 2 日）。白皮书为导览性文档，若与规范文件集冲突，以规范为准。